

정삼각형 분할 열거 알고리즘의 오류 규명 및 정정과 성능 개선에 대한 연구

박진우(한국디지털미디어고등학교)

목차

1. 서론	2. 본론
1.1 연구의 배경	2.1 표준 참조값의 검증과 정정
1.2 연구의 목적	2.2 열거 알고리즘의 성능개선
1.3 선행연구와 그 문제점	3. 결론

정삼각형 분할의 전수 열거는 2010년 Drápal와 Hämäläinen이 크기 20까지 수행한 결과가 표준 참조값으로 쓰여 왔다[4]. 본 연구는 기존 연구의 오류인 꼭짓점 위치만 담은 정준 서명이 서로 다른 분할을 과소집계한 것을 규명 및 정정하여, 크기 16~20의 정정값 19,665 / 66,051 / 224,708 / 771,893 / 2,674,866을 산출하였다. 이 값은 타당성을 인정받아 OEIS A299705의 정정이 승인되었다. 다만 이 오류는 저자들이 처음 발견하였다. 그러나 그 진단은 학술 기록에 이르지 못하고 저장소 안에만 남았으며, 정정값 다섯 중 네 항은 저장소의 이력 어디에도 보이지 않았다. 본 연구는 위 오류의 내용을 독립적으로 밝혀 원저자의 확인을 받았고 그 네 항과 $A(n,k)$ 표를 산출하여 16년간 잘못 쓰여 온 수열을 실제로 바로잡았다.

성능 면에서는 저자들의 C++ 구현에 세 가지 개선을 적용하였다. 선형계 공유 인수분해, 정수 산술 전환, 지연 일괄 정준화를 컴파일 플래그로 이식해 같은 소스와 단일 스레드 조건에서 비교하였다. 하나씩만 적용하면 1.47배/1.44배/2.82배에 그치지만, 세 개선을 함께 적용하면 크기 17에서 82배가 된다. 이 가속으로 출판 범위 너머인 크기 21~23을 열거하여 9,345,070 / 32,876,043 / 116,359,458을 얻었다. 저자들이 그 구간에 남긴 미발표 기록은 separated 개수뿐인데, 본 연구의 separated 개수와 3개 모두 일치하였다.

주제어 : 정삼각형 분할(Equilateral Triangle Dissection), 전수 열거(Exhaustive Enumeration), 동형 판정(Isomorphism Testing), 정준 서명(Canonical Signature), 알고리즘 성능 개선(Algorithm Optimization), OEIS A299705

1. 서론

1.1 연구의 배경

도형을 더 작은 같은 종류의 도형들로 겹치지 않게 채우는 분할 문제는 1940년 Brooks - Smith - Stone - Tutte의 정사각형 분할 연구[2]에서 출발하여, Tutte가 1948년 정삼각형으로 확장하였다[8]. 사용된 조각의 개수를 그 분할의 크기라 할 때, 가장 기본이 되는 질문은 크기가 n 인 분할의 개수이다. 이때 정삼각형의 회전/반사로 겹치는 두 분할은 같은 것으로 센다.

2010년 Drápal와 Hämäläinen이 컴퓨터로 크기 20까지 전수 열거하여 이 값을 처음 제시하였고[4], 그 결과는 OEIS(Online Encyclopedia of Integer Sequences, 온라인 정수열 백과사전)[7]에 A299705로 등록되어 지금까지 표준 참조값으로 쓰여 왔다.

같은 논문은 "nonseparated perfect 분할이 존재하는가?"라는 미해결 문제 하나를 남겼다. 쉽게 말하면 똑같은 조각을 두 번 쓰지 않으면서(perfect), 동시에 조각들이 줄을 맞추지 않고 서로 어긋나게 맞물리는(nonseparated) 분할을 만들 수 있는가를 묻는 것이다(정확한 정의는 2.1.1절). 크기 20까지 전부 찾아보아도 그런 분할은 하나도 나오지 않았으므로, 있다 하더라도 조각이 21개 이상이어야 한다는 것만 알려졌다.

이 상태는 16년째 유지되고 있었다. 열거를 크기 20 너머로 확장한 연구도, 발표된 값을 독립적으로 재계산한 연구도 없었다.

1.2 연구의 목적

1.1절의 미해결 문제도, 개수 자체도 같은 수단에 의존한다. 이 문제에는 n 을 넣으면 답이 나오는 공식이 알려져 있지 않아 가능한 분할을 하나도 빠짐없이 만들어 세는 전수 열거가 유일한 방법이다. 그러나 열거 프로그램의 정확성과 속도 어느 쪽도 그동안 검증된 바가 없었다. 본 연구의 목적은 이 두 가지를 검증하고 개선하는 데 있다.

첫째, 표준 참조값으로 쓰이는 열거 결과가 실제로 맞는지 확인하고, 틀렸다면 바로잡는다. 만약 틀렸다면 이 값을 가져다 쓴 이후의 모든 연구가 함께 틀린 전제 위에 서게 된다.

둘째, 열거 알고리즘을 더 빠르게 만들어 전수 열거가 도달 가능한 크기를 확장한다. 분할의 개수는 크기가 하나 늘 때마다 3배 이상 증가해 크기 20에서 이미 267만 개이고, 크기 21 이상은 공식적으로 발표된 계산이 없다. 이 구간을 세어 보려면 알고리즘의 성능 개선이 필요하다.

1.3 선행연구와의 차이점

1.3.1 정삼각형 분할 문제를 그래프 문제로 치환한 선행연구

정삼각형 분할의 전수 열거 문제를 도형이 아닌 그래프로 접근한 선행연구는 4편이 있었다. Drápal,Kala,Hämäläinen[5]은 분할을 라틴 교환쌍(Latin bitrade)이라는 표로 옮기고, 거꾸로 그 표에서 세운 선형계 $Eq(T,a)$ 를 풀면 조각의 위치와 크기가 되살아남을 보였다. 여기서 라틴 방진은 $n \times n$ 표의 각 행과 열에 기호가 한 번씩 나타나도록 채운 것이다. 라틴 교환쌍(T° , T^*)는 같은 자리들을 차지하는 두 부분표로, 행/열/기호 어느 관점에서 보아도 서로를 정확히 맞바꿀 수 있는 쌍을 말한다.

Cavenagh와 Lisoněk[3]은 크기 n 인 spherical bitrade가 꼭짓점 $n+2$ 개인 평면 Euler 삼각분할(꼭짓점마다 변이 짝수 개)과 일대일로 대응함을 보였다. Brinkmann과 McKay의 plantri[1]는 그 삼각분할을 동형류마다 하나씩 생성한다.

Drápal와 Hämäläinen[4]은 앞의 세 편을 역순으로 이어 실제 열거를 수행하였다. plantri로 삼각분할을 생성하고, bitrade로 옮기고, 어느 꼭짓점을 바깥 삼각형으로 삼을지 바꿔 가며 선형계를 풀어 분할로 되돌린 뒤, 같은 분할이 여러 경로로 거듭 나오므로 마지막에 동형 판정으로 중복을 걸러 낸다. 크기 20까지의 표준 참조값이 여기서 나왔고, 본 연구가 찾은 문제점은 이 마지막 단계, 곧 동형 판정이다.

1.3.2 선행연구의 문제점과 본 연구의 차별점

Drápal와 Hämäläinen[4]이 판정 기준으로 삼은 것은 분할의 꼭짓점 (x,y) 들을 정렬한 목록이다. 이 목록은 분할선들이 만나는 위치는 남기지만 그 사이를 어떤 조각이 채우는지는 남기지 않는다. 그래서 꼭짓점 집합이 같으면서 조각 구성이 다른 분할 쌍이 하나로 계산되어 둘 중 하나가 개수에서 사라지고, 그렇게 작아진 값이 16년간 표준 참조값으로 통용되었다.

이 결함을 저자들이 몰랐던 것은 아니다. 저장소[6]의 2012년 3월 커밋이 꼭짓점 위치만 담은 정준 서명 탓에 놓친 분할들을 짚으며, 그것이 $n=18$ 의 오차 8의 원인이라고 지목해 두었다. 그러나 그 진단은 커밋 메시지와 주석 없는 코드에만 남았고, 정오표도 arXiv 갱신도 OEIS 정정도 없었다.

본 연구의 차별점은 1.2절의 두 목적에 각각 대응한다. 정확성 측면에서는 앞 절의 생성 경로를 그대로 따르되, 꼭짓점 위치가 아니라 조각의 위치, 크기, 방향까지 담은 정준 서명으로 동형 판정 기준을 교체한다. 새 기준이 옳다는 것을 그 기준 자신이 보증할 수는 없으므로, 생성 논리가 전혀 다른 두 열거기로 교차검증한다.

성능 측면에서는 세 곳을 고친다. 바깥 삼각형 후보마다 되풀이하던 소거를 bitrade당 한 번으로 줄이고(공유 인수분해), 그 소거를 유리수 대신 정수로 수행하며(정수 산술 전환), 서명을 디스크로 흘리는 대신 조각 다중집합의 키만 모아 두었다가 마지막에 한 번에 정준화한다(지연 일괄 처리). 앞의 둘은 선형계 풀이 구간에, 나머지 하나는 정준화 구간에 작용한다.

2. 본 론

본 장의 모든 수치는 <표 1>의 환경에서 실험한 결과이다. 2.1절의 결과는 시간이 아니라 개수이므로 장비에 좌우되지 않으나, 실험의 일관성을 위해 이 장비에서 모두 수행하였다.

<표 1> 실험 환경

항목	내용
장비	Apple M3 Max, 16코어, 메모리 64GB
운영체제	macOS
프로그래밍 언어(2.1절)	Python 3.14 + plantri 5.3[1] 열거기 A/열거기 B 모두 순수 파이썬
프로그래밍 언어(2.2절)	g++ -O3 + 단일 스레드

2.1 표준 참조값의 검증과 정정

두 열거기 모두 순수 파이썬으로 구현하였으며, plantri[1]는 열거기 B의 삼각분할 생성에만 쓴다.

2.1.1 기존 열거의 결함

꼭짓점 자리의 목록은 분할선들이 만나는 위치는 남기지만, 그 사이를 어떤 조각들이 채우

는지는 남기지 않는다. 분할의 조각은 바깥 삼각형과 같은 방향인 위 삼각형(▲)과 뒤집힌 아래 삼각형(▼) 두 종류뿐인데, 둘 다 같은 세 방향 직선의 교점을 꼭짓점으로 가진다. 그래서 꼭짓점의 자리만으로는 그 자리에 놓인 조각의 방향도 크기도 결정되지 않는다.

따라서 꼭짓점 집합이 같으면서 조각 구성이 다른 분할 쌍이 원리적으로 존재할 수 있고, 그런 쌍은 과소집계되어 개수에서 사라진다. 문제는 서명이 부정확한 것이 아니라 필요한 정보를 아예 담고 있지 않다는 데 있으므로, 기존 기준을 보완하는 것으로는 해결되지 않는다.

그런 쌍이 아무 분할에서나 나타나는 것은 아니다. 바깥 변에 평행하면서 조각의 변으로 유도되는 직선을 dissecting line이라 할 때, 분할이 separated라는 것은 ① 모든 dissecting line에서 그 위에 놓인 조각 변들의 합집합이 하나의 연결 선분을 이루고 ② 차수 6인 꼭짓점(한 점에 조각 6개가 모임)이 없음을 말한다. 둘 중 하나라도 어기면 nonseparated다. 같은 꼭짓점 위에 조각을 다르게 엮을 여지는 조각 변들이 하나의 선분으로 맞물리지 못하는 자리, 그리고 한 점에 조각 여섯개가 모이는 자리에서 생긴다. 그래서 충돌 쌍이 나온다면 nonseparated 쪽이라고 볼 수 있고, 뒤에서 보듯 오류는 전적으로 nonseparated 쪽에서만 발생하였다. 이는 곧 separated 개수는 서명을 바꾸기 전과 후가 같아야 한다는 뜻이며, 이 성질을 본 연구 구현의 정확성을 판단하는 잣대로 쓴다.

2.1.2 실험 - 새 판정 기준과 두 열거기의 교차검증

가) 새 판정 기준

정삼각형은 전단 변환 $(x,y) \rightarrow (y/2 + x, \sqrt{3} y/2)$ 로 직각삼각형 $\{x \geq 0, y \geq 0, x+y \leq N\}$ 과 대응한다. 이 좌표에서 모든 조각의 변은 가로($y=r$), 세로($x=c$), 대각($x+y=s$)의 세 방향뿐이므로 조각 하나를 (r, c, s) 로 적을 수 있다.

새 판정 기준 (기존 기준: 분할 전체의 꼭짓점 위치만)

조각 하나 ▲ = $\{y \geq r, x \geq c, x+y \leq s\}$, ▼ = $\{y \leq r, x \leq c, x+y \geq s\}$

▲: $(r, c, N-s)$, ▼: $(s-c, s-r, N-r-c)$ 에 방향과 크기를 덧붙임

세 성분은 바깥 세 변에서 조각의 대응 변까지의 거리

모든 조각 표현의 다중집합에 대한, 삼중항 세 성분 6개 치환의 사전순 최소

조각의 변 길이는 $|s-r-c|$ 이고 그 부호가 방향(▲인지 ▼인지)을 결정한다. 즉 (r, c, s) 하나에 조각의 위치, 크기, 방향이 모두 담기며, 기존 기준이 놓친 것이 바로 이 크기와 방향이다. 다만 (r, c, s) 를 그대로 쓰면 대칭이 성분의 치환으로 나타나지 않으므로, ▲는 $(r, c, N-s)$, ▼는 $(s-c, s-r, N-r-c)$ 로 바꿔 적는다. 이 세 성분은 바깥 삼각형의 세 변에서 조각의 대응변까지의 거리이고, 정삼각형의 대칭 6개는 바깥 세 변의 치환이므로 삼중항의 세 성분이 그 치환을 그대로 따라간다. 따라서 모든 조각의 [방향, 크기, 삼중항] 다중집합에 6개 치환을 적용해 사전순 최소를 취하면 정준형이 된다. 계산은 $O(n \log n)$ 이며 전 과정이 인메모리 처리된다.

나) 교차검증을 위한 2개의 열거기

구현에 결함이 있어도 결과는 외견상 타당해 보이기 때문에, 생성 논리가 전혀 다른 두 열거기에 같은 기준을 적용해 결과가 일치하는지 대조하는 교차검증을 실험 설계의 기본으로 삼았다.

1) 열거기 A - 삼각격자 정준 채우기 : 도형을 직접 다룬다. 변 N 인 삼각형을 N^2 개의 단위 칸으로 나누고, 스캔 순서상 아직 덮이지 않은 첫 칸을 찾아 그 칸이 위칸이면 그 칸을 꼭대기로 하는 ▲만, 아래칸이면 그 칸을 맨윗줄 맨왼쪽으로 하는 ▼만 놓는다. 각 조각은 자기가 덮는 칸 중 스캔 순서상 가장 빠른 칸이 정해져 있으므로 이 규칙은 각 분할을 정확히 한 번만 생성하며, 따라서 생성 단계에서 중복이 원천적으로 발생하지 않는다.

2) 열거기 B - 평면 그래프 경우 : 기존 방법[4]의 재구현이다. plantri -b V 로 꼭짓점 $V = n+2$ 인 Euler 삼각분할을 생성한 뒤, 꼭짓점 3색칠 / 면 2색칠로 bitrade를 얻어 조각과 세방향 분할선을 복원하고, 선형계 $Eq(T,a)$ 를 풀어 조각의 크기를 얻는다. $Eq(T,a)$ 는 a 를 바깥 삼각형으로 고정해 $x_{a0}=0$, $x_{a1}=0$, $x_{a2}=1$ 로 두고 나머지 b 마다 $x_{b0} + x_{b1} = x_{b2}$ 를 부과한 것으로, 단순 전파로는 풀리지 않으며 가우스 소거가 필요하다.

두 구현은 서명 함수만 공유한다. 생성 논리에 공통 코드가 없으므로 한쪽의 버그가 다른 쪽에 옮겨갈 수 없고, 두 방법이 같은 값을 낸다면 그 값은 서로 무관한 두 경로를 동시에 통과한 것이 된다.

다) 실험 결과

열거기 B는 $n \leq 15$ 에서 출판값을 완전히 재현하였다($n=4, 6, 7, \dots, 15$ 에 대해 1, 1, 2, 3, 9, 23, 62, 188, 574, 1826, 5953; $n=5$ 는 0이다). 열거기 A는 계산량 때문에 $n \leq 13$ 까지 완주하였고 그 구간에서 열거기 B와 완전히 일치한다. 그러나 $n \geq 16$ 에서 열거기 B의 값이 출판값과 어긋났다.

<표 2> 본 연구의 값과 출판값의 불일치

n	본 연구	출판값	차 (nonseparated)	separated 부분
16	19,665	19,664	+1	15,589 = 15,589
17	66,051	66,049	+2	51,377 = 51,377
18	224,708	224,700	+8	172,162 = 172,162
19	771,893	771,859	+34	583,810 = 583,810
20	2,674,866	2,674,753	+113	1,998,407 = 1,998,407

결정적인 것은 마지막 열이다. separated 개수는 다섯 경우 모두 출판값과 정확히 일치하며, 차이는 nonseparated 부분에만 있다. 2.1.1절의 예측과 부합하는 결과다. (그림 1)이 그 차이를 두 종류로 나누어 보인 것이다.



(그림 1) 출판값에서 누락된 분할 수 (n=14~20)

노란 막대가 separated, 붉은 막대가 nonseparated이며 값이 0인 곳에도 기준선에 얇은 표시를 남겼다. separated는 전 구간 0으로 출판값과 완전히 일치하고, 누락은 n=16부터 nonseparated 쪽에서만 나타나 1, 2, 8, 34, 113으로 늘어난다.

2.1.3 결과의 검증

두 값이 다르다는 것은 둘 중 하나가 틀렸다는 뜻일 뿐이고, 틀린 쪽이 새로 계산한 값일 가능성을 먼저 확인해야 한다. 출판값은 16년간 OEIS의 표준 참조값으로 인용되어 온 값이므로 그것을 뒤집으려면 본 연구의 결과가 옳다는 확실한 증거가 필요하다.

그래서 "우리 값이 다르게 나왔다"에서 멈추지 않고 세 단계로 확인하였다. 첫째, 불일치의 원인이 우리 쪽 결함일 가능성을 하나씩 검사해 배제한다. 둘째, 새 값이 더 크다는 주장이 성립하려면 출판값이 놓친 분할이 실제로 존재해야 하므로, 그 반례를 직접 제시한다. 셋째, 어느 쪽이 옳은지를 넘어 왜 그 차이가 났는지를 확정한다. 추정된 결함을 우리 데이터에 적용해 출판값이 그대로 재현되면, 두 값의 관계가 우연이 아니라 특정 결함의 필연적 귀결임이 드러난다. 세 단계를 모두 통과해야 정정을 주장할 수 있다고 보았고, 이 절의 분량은 그 요구에서 나온 것이다.

가) 자체 오류 가능성의 배제

본 연구의 오류 가능성을 여러 가설과 검사를 통해 확인하였다.

<표 3> 자체 오류 가능성의 배제

가설	검사	결과
서명이 동형을 놓쳐 과다계수	두 독립 정준서명 대조(n=16)	둘 다 19,665
무효한 배치를 생성	19,665개 전부 완전피복 검사	전부 통과
같은 삼각형을 두 번 나열	조각 중복 검사	0건
생성 방법 자체의 버그	열거기 A와 열거기 B 대조	1,604개 완전 일치
파이프라인 전반의 오류	separated 개수 대조(n≤20)	전부 일치

특히 마지막 항목이 중요하다. 같은 코드가 separated는 정확히 재현하면서 nonseparated에 서만 어긋난다는 것은 코드 전반의 오류가 아니라 nonseparated에 특유한 문제임을 뜻한다. 두 방법의 대조는 $n=16$, 바깥변 $N \leq 14$ 구간에서 누적값 1, 15, 39, 76, 192, 424, 714, 857, 1030, 1200, 1604로 완전히 일치하였다.

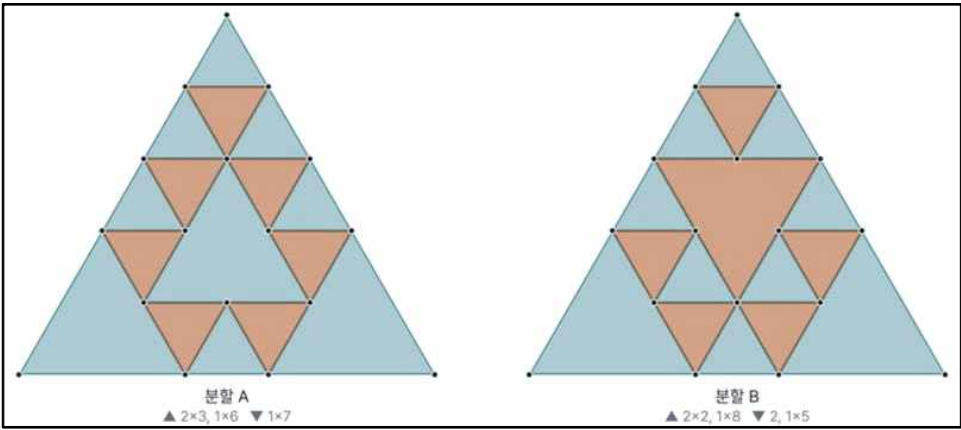
나) 반례의 확인

$n=16$ 에서 꼭짓점 집합이 겹치는 그룹은 정확히 하나이며, 그 안에 서로 다른 분할 두 개가 있다.

<표 4> 꼭짓점 집합이 완전히 겹치는 두 분할($n=16$)

	분할 A	분할 B
바깥 변	5	5
▲ 크기	2×3, 1×6	2×2, 1×8
▼ 크기	1×7	2×1, 1×5
꼭짓점 수	17	17
꼭짓점 집합	완전히 동일	완전히 동일

조각 구성이 명백히 다름에도 17개의 꼭짓점이 하나도 빠짐없이 일치한다.



(그림 2) 크기 16, 바깥 변 5인 두 분할

검은 점은 꼭짓점이며 양쪽이 완전히 동일하다. 분할 B의 가운데에는 ▼2가 있으나 분할 A에는 없다. 꼭짓점 위치만으로 분할을 식별하면 이 둘은 같은 것으로 취급되어 하나가 사라진다.

다) 결함 서명의 재현

반례는 누락이 실재함을 보였을 뿐, 차이 전체를 설명하지는 못한다. 결정적 검증으로, 본 연구가 얻은 분할들에 저자들이 사용한 방식(꼭짓점 위치 집합만)을 그대로 적용하였다.

<표 5> 결함 서명을 적용한 결과와 출판값의 대조

n	올바른 서명	꼭짓점 위치만	과소집계 수 (nonseparated)	출판값
14	1,826	1,826	0	1,826
15	5,953	5,953	0	5,953
16	19,665	19,664	1	19,664
17	66,051	66,049	2	66,049
18	224,708	224,700	8	224,700
19	771,893	771,859	34	771,859
20	2,674,866	2,674,753	113	2,674,753

결함 서명이 출판 범위 전체에서 출판값을 정확히 재현한다. $n \leq 15$ 에서는 과소집계가 없어 출판값이 옳으며, $n=18$ 의 과소집계 8은 저자 커밋의 "off by 8"과 일치한다. 이로써 단순한 값의 불일치가 아니라 그 발생 원인이 확인되었다.

라) 검증 결과의 정리

세 단계가 각각 다른 반론을 막는다. 가)는 서명, 생성, 피복, 중복 어느 쪽에도 결함이 없음을 보여 우리 값이 틀렸을 가능성을 배제하였고, 특히 같은 코드가 separated는 전 구간 재현하면서 nonseparated에서만 어긋난다는 사실이 코드 전반의 오류라는 설명을 무너뜨린다. 나)는 출판값이 놓친 분할을 실물로 제시해 차이가 계산 오차가 아니라 실재하는 누락임을 보였다. 다)는 그 누락이 꼭짓점 위치만 쓰는 서명에서 필연적으로 발생함을 보였다.

특히 다)의 재현은 우연으로 설명되지 않는다. 결함을 가정한 계산이 $n \leq 15$ 에서는 출판값과 어긋나지 않고 $n \geq 16$ 에서만 1, 2, 8, 34, 113만큼 어긋나 일곱 값 모두를 정확히 맞히려면 그 결함이 실제 원인이어야 한다. 여기에 $n=18$ 의 8이 저자 자신의 커밋에 적힌 값과 일치한다는 사실이 더해진다. 세 단계 모두 본 연구의 열거 결과에 오류가 없음을 가리키며, 출판값이 과소계수라는 결론은 그 위에서 성립한다.

2.1.4 정정과 공개 기록화

가) 정정값과 파급 범위

정정값 (OEIS A299705)	
$a(16) = 19665,$	$a(17) = 66051,$
$a(18) = 224708$	
$a(19) = 771893,$	$a(20) = 2674866$

A299705의 20개 항 중 마지막 5개가 모두 과소계수였다. 차이는 1, 2, 8, 34, 113으로 증가하는데, 꼭짓점 집합이 겹치는 분할 쌍이 크기가 커질수록 흔해지기 때문이다.

다섯 정정값 가운데 저자들이 가지고 있던 것은 하나뿐이다. 저장소[6]의 2026년 이전 51개 커밋에 대해 모든 시점의 트리를 검색하면 $n=18$ 의 224708만 나오고, 나머지 네 항 19665/66051/771893/2674866은 어느 시점에도 등장하지 않는다. 즉 이 네 값은 본 연구에서 처음 산출된 것이다.

정정이 다른 결과에 파급되는지 확인하였다. perfect 분할(합동인 조각 쌍이 하나도 없는 분할)의 개수 표가 가장 먼저 확인할 대상이었다. 누락된 분할은 전부 nonseparated였으므로, 그 중 perfect인 것이 있었다면 "nonseparated perfect 분할이 존재한다면 크기가 20을 넘는다"라는 미해결 문제의 전제가 무너지기 때문이다. 올바른 서명으로 재열거한 결과 perfect 동형류

수는 $n = 15$ 부터 2, 2, 6, 23, 64, 181로 논문과 완전히 일치하였고 그중 nonseparated는 어느 n 에서도 0개였다. perfect 표는 서명 결함에도 불구하고 옳았다. 누락된 분할 중 perfect가 하나도 없었기 때문이다.

반면 논문이 개수와 함께 싣고 있는 자기동형군 위수별 분포 $A(n,k)$ 는 같은 구간에서 오류가 있었다. $n = 13, 14, 15$ 에서 논문과 일치함을 확인해 방법을 검증한 뒤 $n \geq 16$ 을 계산하였다.

<표 6> 자기동형군 분포 $A(n,k)$ 의 정정값

n	$A(n,1)$	$A(n,2)$	$A(n,3)$	$A(n,6)$	논문 대비
16	19,380	278	2	5	A2+1
17	65,490	561	0	0	A1+1,A2+1
18	223,630	1,073	0	5	A1+5,A2+3
19	769,875	2,001	8	9	A1+24,A2+9,A6+1
20	2,670,849	4,017	0	0	A1+94,A2+19

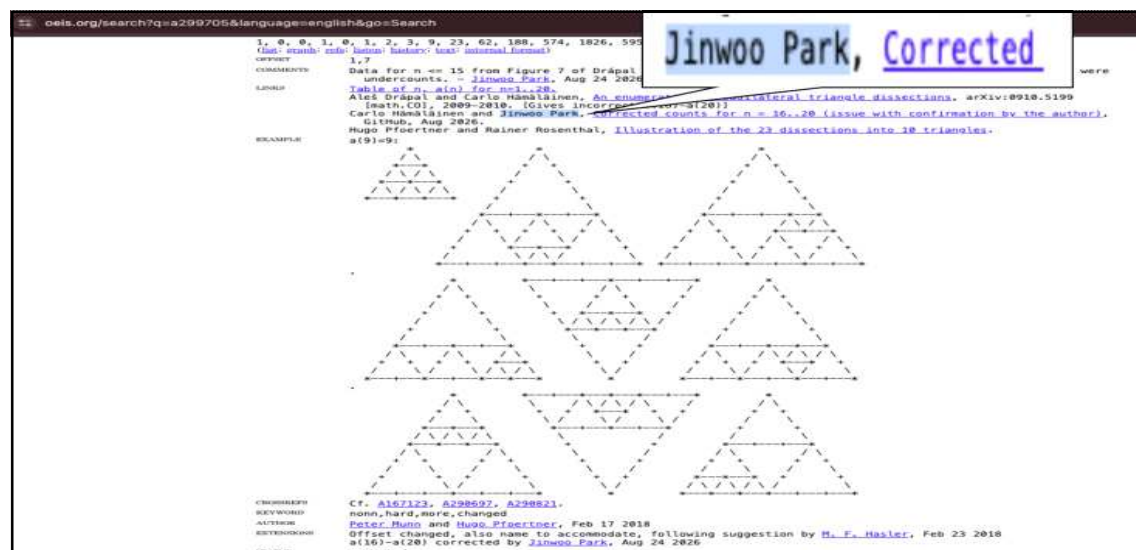
$n=16$ 에서 누락된 분할은 거울대칭을 갖는다. (그림 2)의 분할 A가 바로 그것이다. 그리고 $n=19$ 에서는 위수 6의 완전대칭 분할 하나가 통째로 누락되어 있었다.

나) 원저자의 독립 확인

2026년 8월 본 연구의 결과를 저자들의 공개 저장소에 GitHub 이슈로 보고하였고, 원저자 C. Härmäläinen이 당일 회신하였다. 원저자는 저장소의 수정된 C++ 코드를 직접 재실행하여 다섯 값이 본 연구와 정확히 일치함을 확인하였다. 다섯 값 자체는 열거기 B가 산출한 것이므로, 이로써 서로 독립인 두 계산(본 연구의 열거기 B + 원저자의 코드)이 일치한 것이다. 열거기 A는 계산량 때문에 이 구간까지 확장할 수 없으며, $n \leq 13$ 전 구간과 $n=16$ 의 일부 구간에서 열거기 B의 생성 논리를 검증하는 역할을 맡는다. 원본 코드가 꼭짓점 위치 집합만을 정준 서명으로 사용했다는 진단도 함께 확인되었다.

다) OEIS 수열의 정정

원저자의 확인을 근거로 정정을 제출하였고 승인되었다. 원논문 링크에는 "[Gives incorrect $a(16)-a(20)$]" 라는 주석과 함께 원저자의 확인이 담긴 이슈 링크가 달렸다. separated 수열(A167123)과 perfect 수열(A290653)은 영향이 없어 그대로 두었다.

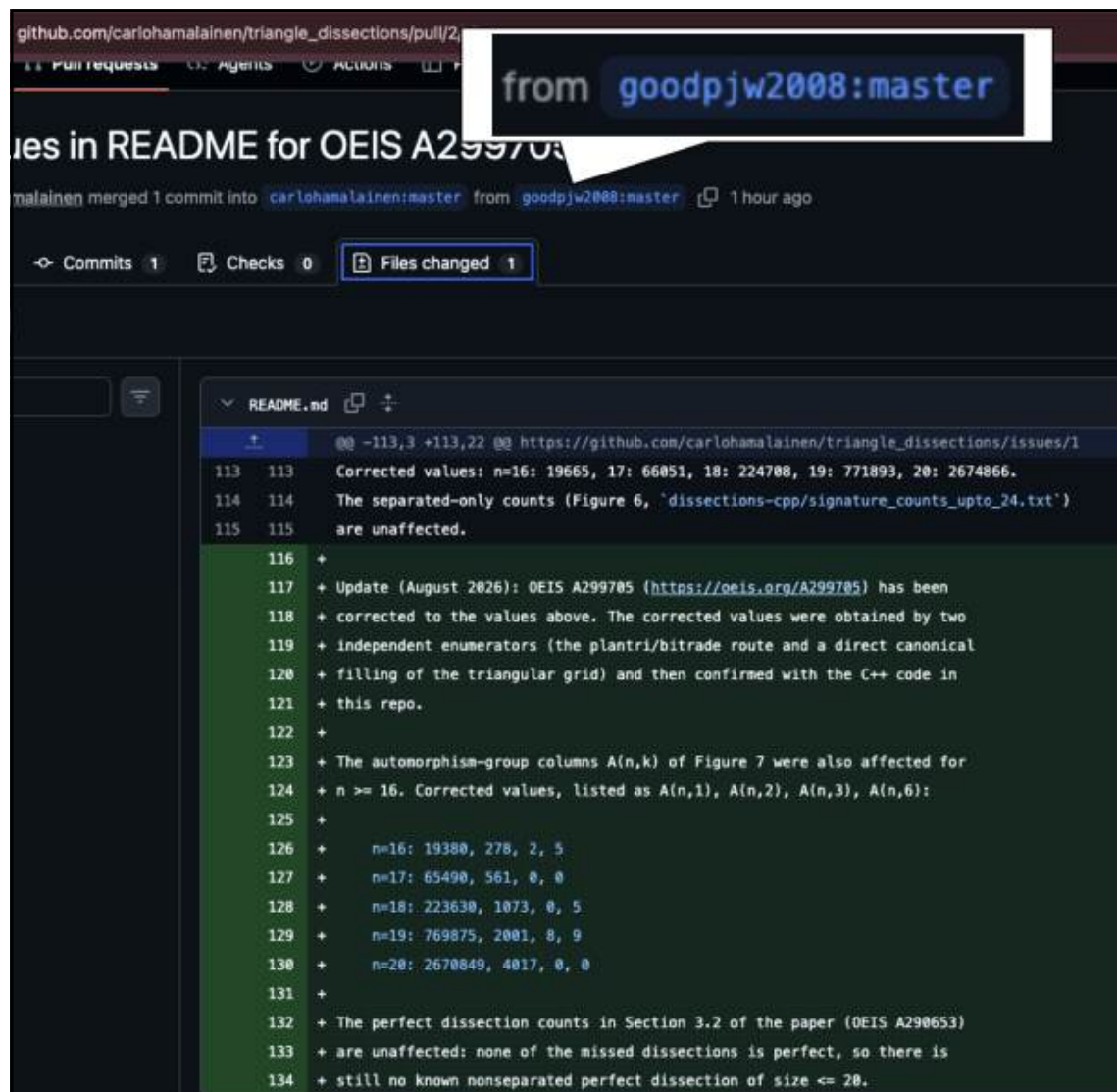


DATA의 마지막 다섯 행이 교체되었고, 원논문 링크에 해당 행이 틀렸다는 주석이 달렸으며, EXTENSIONS에 정정 기여가 기록되었다. STATUS는 approved이다.

16년간 통용된 표준 참조값이 바로잡혔고, 이 주석 덕분에 이후 이 논문을 참조하는 연구가 같은 오류를 물려받지 않는다.

라) 선행연구 저장소 정정

원저자의 초청에 따라 저장소[6]의 README를 정정하는 Pull Request를 보냈고, 원저자가 직접 병합하였다. 정정된 열거 개수와 $n \geq 16$ 의 $A(n,k)$ 열이 함께 반영되어, 정정값이 원저자 자신의 기록 안에 편집되었다.



(그림 4) 원저자가 병합한 Pull Request

README에 정정된 열거 개수와 $n \geq 16$ 의 자기동형군 분포 $A(n,k)$ 가 추가되었고, perfect 분할 개수는 영향이 없다는 설명이 함께 들어갔다.

2.2 열거 알고리즘의 성능 개선

본 절에서는 기존 구현의 병목을 분석하고 성능 향상 방안을 제시한다.

2.2.1 기존 구현의 병목

기존 방법을 그대로 따르기 전에 더 단순한 접근 두 가지를 먼저 검토하였다. 직접 탐색(조각 크기의 조합을 디오판토스 방정식으로 직접 푸는 방식)은 후보 수가 단계당 100~250배로 늘어 $n=21$ 에서 10^{17} 규모가 되므로 불가능하고, 동적계획법이 성립하려면 부분 채움을 상수 개의 매개변수로 요약한 상태로 묶을 수 있어야 하는데, 실제 분할의 부분 채움은 경계가 불록 육각형이 아니라 계단 모양이어서 그런 파라미터화가 존재하지 않는다(크기 11까지의 전 분할에서 반례를 확인). 따라서 기존 파이프라인 자체를 가속하는 것 외에 방법이 없다.

개선의 목표는 열거 결과를 바꾸지 않은 채 도달할 수 있는 크기를 확장하는 것이다. 열거기의 출력은 표준 참조값 자체이므로, 근사/표본추출/해시 충돌을 감수하는 기법은 배제하고 같은 답을 더 적은 연산으로 얻는 변환만 채택하였다.

원 구현을 프로파일하면 시간은 두 구간으로 갈린다. 선형계 풀이가 35%, 조각 서명의 생성/정렬/중복 제거가 65%($n=17$ 기준). 각 구간에서 제거할 수 있는 병목은 다음 셋이다.

- 1) **같은 소거의 반복** : 바깥 삼각형 후보를 하나 고를 때마다 방정식 행렬을 새로 만들어 소거를 되풀이한다. 그러나 그 행렬은 후보와 무관하다.
- 2) **유리수 산술** : 소거 전 과정을 유리수로 수행한다. 분모, 분자가 함께 커지며 연산 비용이 불어난다.
- 3) **값비싼 서명과 디스크 경유** : 조각을 유리수 점으로 되돌려 6개 대칭상을 모두 만든 서명을 디스크에 기록한 뒤, 외부 정렬로 중복을 제거한다.

2.2.2 실험 - 단계별 개선과 성능 측정

가) 측정 규약

성능 개선율은 장비, 언어, 컴파일러에 좌우되므로 다섯 변형을 모두 같은 옵션으로 빌드하고, 저자들의 `spherical_trades_binary`가 만든 동일한 `bitrade` 스트림을 입력으로 주었다. 시간은 3회 실행 후 최솟값을 취하였다.

측정은 전부 단일 스레드다. 병렬화는 알고리즘을 바꾸는 것이 아니라 CPU 자원을 늘리는 것이어서 개선 효과와 섞이면 안 되기 때문이다.

나) 비교 방법

기존 연구와 본 연구를 같은 언어에서 비교하기 위해, 저자들의 `td.cpp`를 한 소스로 두고 세 개선을 컴파일 플래그로 켜고 끄는 방식을 택하였다. 플래그가 꺼진 경로는 원저자 코드 그대로이므로, 두 구현의 차이는 플래그로 활성화되는 코드뿐이다. 입력 파싱/조각 복원/서명 계산은 원저자 코드를 그대로 재사용하였다.

실험 시나리오. 기존 연구를 고정된 기준으로 두고, 본 연구가 적용하는 개선만 바꾸어 가며 <표 7>의 네 가지를 비교한다.

<표 7> 실험시나리오

시나리오	기존 연구	본 연구	무엇을 보는가
1	원 구현 그대로	1단계만 적용	선형계 풀이 (공유 인수분해의 단독 효과)
2	원 구현 그대로	2단계만 적용	선형계 풀이 (정수 산술 전환의 단독 효과)
3	원 구현 그대로	3단계만 적용	정준화 (지연 일괄 처리의 단독 효과)
4	원 구현 그대로	1+2+3 모두 적용	세 개선을 함께 적용 효과

시나리오 1~3에서 커지 않은 개선의 자리에는 원저자 코드가 그대로 돌아간다. 1,2단계는 선형계 풀이 구간에, 3단계는 정준화 구간에 작용한다. 이 대응이 마)의 분석에서 그대로 쓰인다. 비교가 성립하려면 모든 변형이 같은 일을 해야 한다. 세 가지로 확인하였다.

- 1) 플래그를 모두 끈 빌드는 원본 td와 시간이 일치한다(n=17에서 두 실행 모두 17.2초 수준)
- 2) 1단계, 2단계 변형이 출력한 서명 집합은 원본과 바이트 단위로 동일하다($n \leq 16$ 전수 비교)
- 3) 다섯 변형 모두 정확히 같은 수의 해를 처리한다(n=17에서 155,822개)

다) 개선 기법

- 1) 1단계 - 선형계 풀이 (공유 인수분해) : 원 구현은 바깥 삼각형 후보 a 마다 방정식 행렬을 새로 만들어 RREF를 반복한다. 그러나 $\text{Eq}(T,a)$ 에서 a 에 의존하는 것은 $x_{a0}=0, x_{a1}=0, x_{a2}=1$ 의 세 고정 조건뿐이고, T° 전체의 방정식 행렬 M 은 a 와 무관하다. 따라서 $[M \mid I]$ 를 bitrade 당 한 번만 소거해 두고 a 마다는 2×2 경계조건만 풀면 된다. 소거 횟수가 $|T|$ 번에서 한 번으로 준다.
- 2) 2단계 - 선형계 풀이 (정수 산술 전환) : 행 연산이 해공간을 보존한다는 점을 이용해 소거 전체를 정수로만 수행하고(행마다 gcd 정규화로 성분 팽창 억제) 해를 (정수벡터, 공통 분모) 쌍으로 반환한다.
- 3) 3단계 - 정준화 (지연 일괄 처리) : 원 구현은 조각을 유리수 점으로 복원해 6개 대칭상을 모두 만든 서명을 디스크에 흘린 뒤 외부 정렬로 중복을 제거한다. 본 연구는 스트리밍 중에는 조각 다중집합의 정확한 키만 등록하고, 마지막에 서로 다른 키에 대해서만 6개 치환 변형을 만들어 최소화한다. 근사 해시가 아니라 데이터 전체가 키이므로 정확성 손실이 없고, 전 과정이 디스크를 거치지 않는다.

라) 실험 결과

네 시나리오의 결과는 <표 8>과 같다. 각 칸은 기존 연구 $\rightarrow$ 본 연구의 실행 시간이고 괄호 안이 배속이다.

<표 8> 시나리오별 기존 연구 대비 본 연구 (C++, 단일 스레드, 3회 중 최솟값)

시나리오	본 연구의 개선	n=16	n=17	n=18
1	1단계	5.11 → 3.63s (1.41배)	17.20 → 11.73s (1.47배)	66.71 → 43.59s (1.53배)
2	2단계	5.11 → 3.69s (1.38배)	17.20 → 11.91s (1.44배)	66.71 → 44.37s (1.50배)
3	3단계	5.11 → 1.65s (3.10배)	17.20 → 6.09s (2.82배)	66.71 → 25.30s (2.64배)
4	1+2+3	5.11 → 0.06s (85.2배)	17.20 → 0.21s (82배)	66.71 → 0.83s (80.4배)



(그림 5) 기존 연구와 본 연구의 실행 시간 대조 (n=17)

기준선 17.20초는 네 시나리오에서 모두 같고, 위쪽 숫자는 기존 연구 대비 배속이다.

개선 하나씩만 켜 시나리오 1~3은 1.4~3.1배에 그치지만, 셋을 함께 켜 시나리오 4는 세 크기 모두에서 80배를 넘는다. 그 이유는 마) 에서 분해한다. 네 시나리오의 열거 결과는 모두 19,665 / 66,051 / 224,708로 기준 빌드와 완전히 일치한다. 여기서 기준이 되는 저장소의 C++ 코드는 저자들이 2012년에 서명을 고친 뒤의 것이어서 2.1절의 정정값을 낸다. 즉 2.2절은 출판값이 아니라 수정된 구현을 상대로 속도만 비교한 것이다.

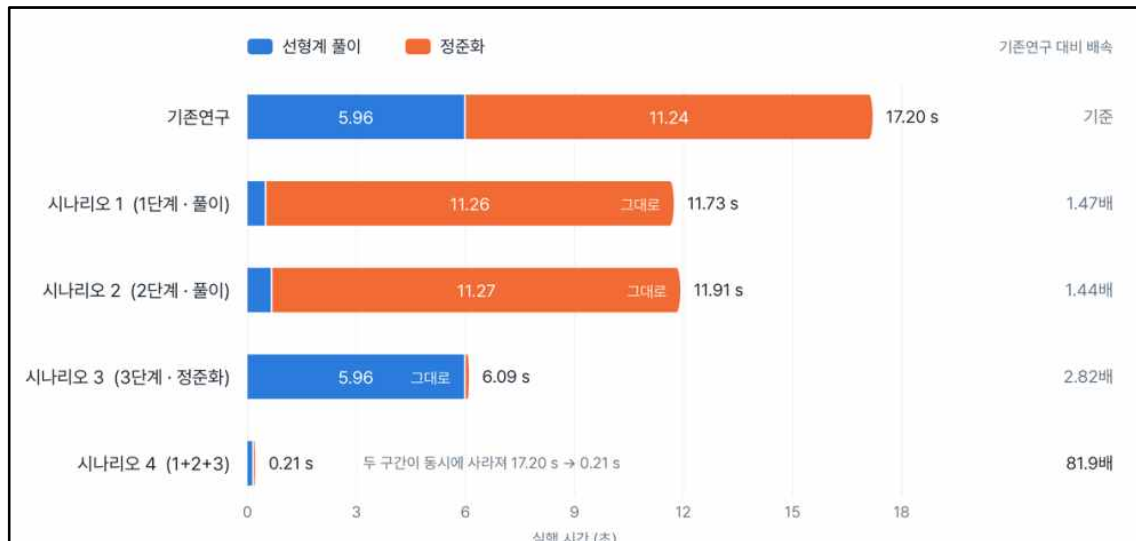
마) 결과 분석 - 시나리오 4는 왜 80배를 넘는가

개별 배율의 곱은 $1.47 \times 1.44 \times 2.82 \approx 6$ 배인데 실제로 셋을 함께 적용 시 82배이다. 원인을 확인하기 위해 정준화 단계를 끈 빌드로 선형계 풀이 시간만 따로 측정하였다.

<표 9> 실행 시간의 구간 분해 (n=17)

구현	선형계 풀이 (기준 35%)	정준화 (기준 65%)	합계
기준 연구	5.96s	11.24s	17.20s
시나리오 1 (1단계)	0.47s	11.26s (그대로)	11.73s
시나리오 2 (2단계)	0.64s	11.27s (그대로)	11.91s
시나리오 3 (3단계)	5.96s (그대로)	0.13s	6.09s

풀이 시간은 정준화를 끈 빌드로 직접 측정하였고, 정준화 열은 전체에서 그 값을 뺀 것이다. 같은 내용을 (그림 6)에 표현했다.



(그림 6) 시나리오별 실행 시간의 구간 분해 (n=17)

파란 구간이 선형계 풀이, 주황 구간이 정준화다. 시나리오 1, 2는 정준화 11.26초가, 시나리오 3은 풀이 5.96초가 손대지 않은 채 남아 상한을 잡는다. 시나리오 4에서만 두 구간이 동시에 사라져 막대가 거의 남지 않는다.

1, 2단계는 풀이 구간에만 작용한다. 그 구간은 전체의 35%(5.96/17.20)뿐이므로 풀이를 0초로 만들어도 상한이 $17.20/11.24 = 1.53$ 배다. 실제로 풀이는 12.7배 빨라졌는데도(5.96 → 0.47초) 전체는 1.47배에 그친다. 3단계도 동일하다. 정준화를 0.13초로 줄였지만 남은 풀이 5.96초 때문에 상한이 2.89배로 제한된다. 암달의 법칙이다.

두 구간은 시간이 더해지는 관계이므로 개별 배율을 곱하는 것은 의미가 없다. $T = A + B$ 일 때 개별 가속은 손대지 않은 항이 남은 시간의 하한이 되지만, 둘 다 줄이면 그 하한 자체가 사라진다.

$$(5.96+11.24)/(0.47+11.26) = 1.47, (5.96+11.24)/(5.96+0.13) = 2.82, \\ (5.96+11.24)/(0.13+0.08) = 82$$

시나리오 4가 80배를 넘는 이유는 이것이다. 셋을 함께 적용하면 상한을 잡을 구간 자체가 남지 않는다. 17.20초가 0.21초가 되는 것은 두 구간이 동시에 사라진 결과이지 개별 효과가 곱해진 것이 아니다.

즉 세 개선은 서로를 필요로 한다. 어느 하나만으로는 남은 병목에 막히고, 함께 적용해야 두 병목이 동시에 사라진다.

2.2.3 도달 범위 - $n = 21 \sim 23$ 의 열거

1.2절의 목적 2는 속도 자체가 아니라 도달할 수 있는 크기의 확장이었다. 본 절은 개선된 구현이 실제로 출판 범위 너머에 도달하는지 확인한다. 시나리오 4의 구현을 그대로 쓰며, 컴파일 옵션과 단일 스레드 조건도 2.2.2절과 같다. 다만 실행이 길어 각 크기를 한 번씩만 측정하였다.

<표 10> 출판 범위 너머의 열거 (시나리오 4 구현, 단일 스레드)

n	시간	최대 메모리	전체	separated
20	12.42 s	1.0 GB	2,674,866	1,998,407
21	48.16 s	4.1 GB	9,345,070	6,902,289
22	194.69 s	14.4 GB	32,876,043	24,049,989
23	785.06 s	46.8 GB	116,359,458	84,364,122

$n=20$ 은 2.1절의 정정값을 그대로 재현한다. 시간과 메모리는 전체 열거 실행의 값이고, separated 열은 같은 구현을 --separated로 다시 돌려 얻었다.

두 열의 성격이 다르다. 저자들이 출판 범위 너머에 남긴 기록은 separated뿐이다. 저장소 [6]의 signature_counts_upto_24.txt 와 미발표 개정 원고에는 크기 24까지의 separated 개수가 들어 있으나, 같은 구간의 전체 개수는 저장소 어디에도 없다. 원고의 전체 개수 표조차 크기 20에서 끝난다. 이 자료는 논문에도 OEIS에도 실린 적이 없다. separated 수열 A167123[7] 역시 크기 20까지만 등재되어 있다.

새로 산출한 값 (어디에도 없던 값)

$$a(21) = 9345070, \quad a(22) = 32876043, \quad a(23) = 116359458$$

따라서 검증과 산출이 같린다. separated 열은 검증 자료다. 본 연구의 값이 세 크기 모두 저자들의 기록과 정확히 일치하며, 이는 그 구간에서 구현이 옳게 동작한다는 근거다. 그 위에서 얻은 전체 개수 세 값이 새로 산출된 결과다. 방향을 뒤집으면, 저자들의 미발표 separated 값 세 개도 이로써 독립 구현의 확인을 받았다.

전체 개수 쪽에는 대조가 하나 더 있다. 본 연구는 같은 알고리즘을 파이썬으로도 구현해 두었는데, 삼각분할 생성에는 양쪽 모두 plantri[1]를 쓰지만 bitrade 변환, 선형계 풀이, 정준화는 공유하는 코드가 없다. 두 구현이 $n=21, 22$ 에서 같은 전체 개수를 냈다. 다만 이것도

2.1절의 두 열거기처럼 생성 논리 자체가 다른 교차검증은 아니므로, 이 구간의 값은 아직 그만한 보증을 받지 못한 상태다.

본 실험 환경(64GB)에서 먼저 한계에 닿은 쪽은 시간이 아니라 메모리였다. 크기가 하나 늘 때 시간은 3.9~4.0배, 메모리는 4.1배에서 3.3배로 늘어 증가 속도 자체는 비슷하지만, 이 장비에서는 메모리가 먼저 소진되었다. 자연 일괄 정준화가 서로 다른 정확한 키를 전부 메모리에 모았다가 마지막에 한 번 정준화 하기 때문이며, 이 방식이 3단계 속도의 근거인 동시에 도달 범위의 제약이기도 하다.

$n=23$ 이 46.8GB로 본 환경에서 가능한 마지막 크기이며, 마지막 비율로 추정하면 $n=24$ 는 152GB 규모다. 이는 알고리즘 자체의 한계라기보다 본 실험에 사용한 장비의 한계이므로, 메모리가 더 큰 장비에서는 같은 구현으로 몇 크기 더 나아갈 수 있다. 다만 시간도 크기당 약 4배로 늘기 때문에 그 확장이 무한히 이어지지는 않는다. 메모리를 충분히 확보하더라도 어느 지점부터는 시간이 제약이 된다.

3. 결 론

지금까지 정삼각형 분할의 전수 열거를 서로 독립적인 두 방법으로 재구현하여 출판값과 대조하고, 그 결과를 세 단계로 검증하였으며, 열거 알고리즘을 세 단계로 개선하여 성능을 측정하였다. 그리고 본 연구가 내린 결론은 2가지다.

(1) 16년간 표준 참조값으로 쓰여 온 열거 결과는 마지막 다섯 항이 틀렸으며, 정정값은 19,665 / 66,051 / 224,708 / 771,893 / 2,674,866 이다.

$n \leq 15$ 에서는 출판값을 완전히 재현하였으나 $n = 16 \sim 20$ 에서 각각 1, 2, 8, 34, 113만큼 큰 값을 얻었다. 어긋난 쪽이 본 연구일 가능성을 다섯 가지 검사로 먼저 배제하였다.

원인은 동형 판정의 정준 서명이 분할의 꼭짓점 위치만을 담아, 꼭짓점 집합이 같고 조각 구성이 다른 분할 쌍이 과소집계된 것이다. 결정적 증거는 재현이다. 저자들이 쓴 결함 서명을 본 연구의 데이터에 그대로 적용하자 $n = 14 \sim 20$ 의 출판값 일곱 개가 정확히 재현되었다. 이어 원저자 C. Härmäläinen이 자신의 C++ 코드로 다섯 값을 재계산해 본 연구와 정확히 일치함을 확인하였다. 서로 독립인 두 계산이 같은 값을 낸 것이다.

이를 근거로 OEIS A299705의 정정이 승인되었고, 원저자가 직접 병합한 Pull Request로 정정값이 그의 저장소에도 남게 되었다. 정정의 과급은 줄아, separated 개수와 perfect 분할 표는 영향을 받지 않았고 자기동형군 분포 $A(n,k)$ 만 함께 정정하였다.

다만 버그의 최초 발견자는 저자들 자신이다. 2011년에 열거 결과를 의심해 미발표 개정 원고의 표 캡션을 "하한"으로 강등하였고, 2012년 3월 커밋에서 정준 서명이라는 실제 원인과 $n=18$ 의 오류를 규명하였다. 본 연구가 도달한 것과 같은 진단이며, 시점은 14년 앞선다. 그러나 그 진단은 커밋 메시지와 설명 없는 코드 파일에만 남았고, 강등했던 캡션마저 2012년 5월에 원래대로 되돌려져 최종 상태에는 오류 표시가 남지 않았다.

따라서 본 연구는 최초 발견을 주장하지 않는다. 다만 다음 세 가지는 본 연구의 기여로 주장한다.

첫째, 저자들의 진단을 미리 알지 못한 상태에서 독립적인 경로로 같은 결함에 도달해 그것이 실재함을 재확인하였다.

둘째, 저자들의 저장소 전 이력에도 존재하지 않는 정정값 네 항($n=16, 17, 19, 20$)과

$A(n,k)$ 의 정정 표를 실제로 산출하였다.

셋째, 16년간 커밋 메시지와 주석 없는 코드에만 간혀 있던 사실을 OEIS 정정과 저장소 병합을 통해 학술 기록으로 남겼다. 결함을 알고 있는 것과 표준 참조값을 실제로 바로잡아 놓는 것은 다른 일이며, A299705의 마지막 다섯 항이 옳은 값이 된 것은 이 세 번째 단계를 거친 뒤다.

(2) 열거 알고리즘은 결과를 바꾸지 않은 채 약 80배 가속할 수 있다. 단 세 개선을 함께 적용해야 한다.

기존 연구의 C++ 구현과 같은 언어/같은 소스/같은 컴파일러/단일 스레드로 비교하였다. 저자들의 td.cpp에 세 개선을 컴파일 플래그로 넣어, 플래그를 끄면 원본 코드가 그대로 돌게 하였다. $n=17$ 에서 기존 연구 17.20초가 셋을 모두 적용하면 0.21초(82배)가 된다. $n=16$, 18에서도 85.2배, 80.4배로 안정적이다.

개선 하나씩만 켜면 1.47배/1.44배/2.82배에 그친다. 시간이 선형계 풀이와 정준화라는 두 구간의 합이라서, 한쪽만 줄이면 남은 쪽이 상한을 잡기 때문이다(암달의 법칙).

속도를 정확성과 맞바꾸지 않았음도 확인하였다. 다섯 변형이 모두 같은 수의 해를 처리하고 같은 개수를 내며, 1, 2단계 변형의 서명 집합은 원본과 바이트 단위로 동일하다.

이 가속으로 목적 2에서 겨냥한 범위에 실제로 도달하였다. 같은 장비에서 단일 스레드로 $n = 21/22/23$ 을 각각 48초/195초/785초에 열거하여, 어디에도 없던 전체 개수 9,345,070 / 32,876,043 / 116,359,458을 얻었다. 이 세 값은 저자들의 미발표 기록에도 없던 것이다.

같은 실행의 separated 개수가 그들의 미발표 표와 세 크기 모두 일치하여 구현의 근거가 되었다. 다만 크기가 하나 늘 때 메모리가 3.3배 이상으로 늘어 $n=23$ 의 46.8GB가 64GB 환경의 한계였다. 본 실험 환경에서 먼저 걸린 것은 시간이 아니라 메모리였고, 더 큰 n 을 겨냥한다면 정준화를 외부 저장으로 옮기는 설계가 필요하다. 또한 새로 확장한 범위의 값은 아직 앞서와 같은 수준의 독립적인 검증을 거치지 못했다. 확장된 범위에도 같은 수준의 검증을 갖추는 일, 그리고 그 범위를 남아 있는 미해결 문제의 진전으로 잇는 일을 후속 과제로 남긴다.

끝으로 이 사례가 남기는 교훈은, 전수 열거에서 가장 취약한 지점이 생성이 아니라 동형 판정이라는 것이다. 생성의 오류는 개수를 크게 어긋나게 해 곧 드러나지만, 동형 판정의 결함은 소수의 대상만 눈에 띄지 않게 과소집계하므로, 작은 입력에서의 일치는 동형 판정의 정확성을 보장하지 않는다. 실제로 $n \leq 15$ 에서는 과소집계가 0이었다.

참고문헌

- [1] G. Brinkmann, B. D. McKay, plantri (software), version 5.3, users.cecs.anu.edu.au/~bdkm/plantri, 2022.
- [2] R. L. Brooks, C. A. B. Smith, A. H. Stone, W. T. Tutte, "The dissection of rectangles into squares", *Duke Mathematical Journal*, Vol. 7, pp.312-340, 1940.
- [3] N. J. Cavenagh, P. Lisoněk, "Planar Eulerian triangulations are equivalent to spherical Latin bitrades", *Journal of Combinatorial Theory Series A*, Vol. 115, No. 1, pp.193-197, 2008.
- [4] A. Drápal, C. Härmäläinen, "An enumeration of equilateral triangle dissections", *Discrete Applied Mathematics*, Vol. 158, No. 14, pp.1479-1495, 2010.
- [5] A. Drápal, V. Kala, C. Härmäläinen, "Latin bitrades, dissections of equilateral triangles and abelian groups", *Journal of Combinatorial Designs*, Vol. 18, No. 1, pp.1-24, 2010.
- [6] C. Härmäläinen, Triangle dissection enumeration code, github.com/carlohamalainen/triangle_dissections, 2012.
- [7] N. J. A. Sloane (ed.), *The On-Line Encyclopedia of Integer Sequences*, sequences A299705, A167123, A290653, oeis.org.
- [8] W. T. Tutte, "The dissection of equilateral triangles into equilateral triangles", *Proceedings of the Cambridge Philosophical Society*, Vol. 44, pp.463-482, 1948.

박진우

이메일 : goodpju2008@gmail.com